libkdl: A Kernel Dynamic Linker for Heterogeneous GPU Dispatch via MLIR

mlir-hetero-dispatch

S Akash

IIT Patna | CERN GSoC | vLLM Contributor

LLVM Developers' Meeting — Dublin 2026

The Problem: MLIR Compiles Multi-Target, But Cannot Dispatch

- MLIR's `gpu-module-to-binary` **already compiles** a single kernel to NVPTX + AMDGCN + SPIR-V + x86
- But `#gpu.select_object` resolves at **compile time**
- No upstream mechanism queries runtime hardware to choose among variants

The Missing Piece

`ld.so` resolves shared libraries at runtime.
Who resolves GPU kernels?

Real Deployments

- Cloud: mixed NVIDIA/AMD fleets
- HPC: Frontier (AMD), Aurora (Intel), Perlmutter (NVIDIA)
- Edge: unknown hardware at deploy time
- Serving: multi-tenant, mixed GPU generations

Why Runtime Dispatch Matters

"ML Kernels Are Static" Is False

1. **cuBLAS**: Hundreds of GEMM kernels per precision; ML-trained recommender selects at runtime
2. **cuDNN v9**: NVRTC-compiles fusion kernels on-the-fly based on compute capability
3. **PyTorch dispatcher**: Computes `DispatchKeySet` on *every* operator call

Where Runtime Dispatch Adds Value

4. **torch.compile**: Guards checked per invocation; shape changes trigger recompilation
5. **vLLM**: Piecewise CUDA Graphs with runtime batch-size bucket selection
6. **CUDA Graphs**: Each shape requires re-recording — multi-version dispatch in practice

Helix (ASPLOS 2025): 3.3× throughput on heterogeneous GPU clusters

Survey: Compile-Time Portability Frameworks

ALPAKA (Helmholtz/CERN)

- Header-only C++20, Redundant Hierarchical Parallelism
- CUDA, HIP, SYCL, OpenMP, TBB backends
- >**94%** of native CUDA; CMS Run 3 production
- No runtime dispatch

HIP (AMD)

- Thin header-swap: zero runtime overhead
- ∼90–95% automatic CUDA translation
- Compile-time target only

Kokkos (Sandia/DOE)

- Polymorphic memory layouts + execution policies
- **P3 scores 0.75–0.99** (highest measured)
- <5% overhead vs. native
- Compile-time backend selection

RAJA (LLNL)

- Lambda-based execution policies
- P3 scores 0.47–1.00, <3% overhead
- Compile-time only; no memory mgmt

Survey: Runtime Portability Layers

SYCL / DPC++ (Khronos)

- Runtime device selection via `device_selector`
- P3 scores **0.46–0.65** — poor portability
- SYCL-MLIR: up to **4.3**× speedup (CGO 2024)
- No MLIR production integration

AdaptiveCpp (SSCP)

- Single-Source Single-Compiler Pass
- Runtime JIT: LLVM IR → PTX/SPIR-V/amdgcn
- ∼15% first-launch overhead, near-zero cached
- **Closest prior art** to our approach

OpenCL (Khronos)

- Source portability ≠ performance portability
- NVIDIA never moved beyond 1.2; key legacy: SPIR-V
- Declining ecosystem

Vulkan Compute

- Broadest reach (NVIDIA, AMD, Intel, ARM)
- RDNA3: **matches or exceeds** ROCm
- IREE SOTA on AMD via Vulkan/SPIR-V

Survey: Compiler-Based Systems

Triton (OpenAI)

- Python DSL → 9 MLIR dialects → PTX/AMDGCN
- Matches or exceeds cuBLAS
- Most MLIR-native ML compiler
- No runtime dispatch; no Intel/CPU

TVM (Apache)

- MetaSchedule auto-tuning
- Near-cuBLAS after hours of tuning
- Untuned: 2–5× slower
- No runtime dispatch; uncertain future

XLA / StableHLO (Google)

- First-class MLIR via StableHLO dialect
- PJRT hardware-agnostic runtime API
- Near-native for fused workloads
- No dynamic backend selection

IREE (Google/OpenXLA)

- Full MLIR compiler + runtime
- SOTA Llama2/SD on AMD RDNA3 via Vulkan
- Issue #50 (2019): target config **unsolved 6+ years**
- Issue #12230: runtime selection "**sort of broken**"

Survey: JIT-Based Systems

OCCA (DoE/Shell)

- OKL (annotated C) compiled at runtime
- CUDA, HIP, SYCL, OpenCL, OpenMP, Metal
- Can outperform RAJA on structured grids (6.07s vs 8.02s)
- Non-standard language; no ML adoption

Proteus (LLNL, CGO 2025)

- Portable JIT via embedded LLVM IR
- **2.8**× on AMD, 1.78× on NVIDIA vs AOT
- 1.23× better than CUDA Jitify
- One vendor at a time; high JIT cost

Multi-Versioned SGEMM

- Within **10%** of theoretical peak
- Demonstrates value of per-arch specialization
- arXiv:2507.15277

JIT + Autotuning (2025)

- >**230%** improvement for LLM kernels
- Runtime specialization essential for ML workloads
- arXiv:2505.03780

Key insight: JIT works but has latency. AOT + runti

Comparison Matrix: 14 Systems Across 5 Categories

System	NVIDIA	AMD	Intel	CPU	Runtime	MLIR	Perf
ALPAKA	CUDA	HIP	SYCL	OMP	No	None	>94%
Kokkos	CUDA	HIP	SYCL	OMP	No	None	>95%
RAJA	CUDA	HIP	Exp.	OMP	No	None	>97%
HIP	nvcc	Native	—	—	No	None	100%
SYCL/DPC++	Plugin	Plugin	Native	Yes	Selector	None	85–100%
AdaptiveCpp	CUDA	HIP	SPIR-V	OMP	JIT	None	∼85%
Vulkan	Yes	Yes	Yes	—	SPIR-V	Via SPIR-V	70–100%
Triton	CC 8.0+	ROCm	—	Exp.	No	Native	∼100%
TVM	CUDA	ROCm	OCL	LLVM	No	Shallow	∼100%
XLA	NVPTX	ROCm	Dev	Yes	No	StableHLO	∼100%
IREE	CUDA	HIP	Vulkan	ARM	First-match	Full	90–100%
OCCA	CUDA	HIP	DPC++	OMP	JIT	None	Varies
Proteus	CUDA	HIP	—	—	JIT/vendor	None	+2.8×
libkdl (Ours)	**NVPTX**	**AMDGCN**	**SPIR-V**	**x86**	**Cost-model**	**Native**	**>90%**

SPIR-V: Strengths and Limitations

Strengths

- Broad hardware reach via Vulkan ecosystem
- IREE achieves **SOTA on AMD RDNA3** via SPIR-V
- Vulkan **matches or exceeds ROCm** on RDNA3 (llama.cpp)
- `VK_KHR_cooperative_matrix` enables tensor core access
- MLIR has first-class SPIR-V dialect

Limitations

- **50–80%** of native performance [arXiv:2603.28793]
- NVIDIA: ∼20–30% slower than CUDA for LLM workloads
- No portable representation for vendor-specific intrinsics (tensor cores, MFMA)
- Vulkan capability queries lack compute-specific details (SM count, warp size)

Our Approach

Use SPIR-V as **fallback**, not primary. Prefer native backends (cubin, hsaco) when

How Production ML Systems Actually Dispatch

System	Dispatch Mechanism	Key Detail
cuBLAS	ML-trained recommender	93% of optimal; runtime SM count override
cuDNN v9	3 heuristic modes + NVRTC JIT	On-the-fly fusion kernels by compute cap
PyTorch	`DispatchKeySet` per op call	Live tensor metadata + thread-local state
torch.compile	Guard checks per invocation	Shape change → recompilation or fallback
vLLM	CUDA Graph bucket selection	Piecewise graphs; hetero GPU **unsupported**
ONNX RT	Execution Provider chain	Priority-ordered fallback (CUDA → CPU)

Every major ML system does runtime dispatch.
None does it *across vendors* from a *single MLIR artifact*.

The Gap: What No Existing System Provides

Gap 1: No MLIR Runtime Dispatch

`gpu.select_object` is compile-time only.
IREE #12230: runtime selection "sort of broken."
IREE #15334: all multi-versioning tasks **unchecked**.

Gap 2: No Lightweight Layer

- IREE: 100K+ LOC, full VM + HAL
- Upstream MLIR: fixes target at JIT time
- **Nobody built the <1000 LOC shim**

Gap 3: No Cost-Model Selection

IREE: first-valid-match wins.
CUDA fatbin: best SM-match wins.
No cross-vendor roofline-driven selection.

Gap 4: No Cross-Vendor Fat Binary

CUDA fatbin, HIP bundle, LLVM offload binary each handle one vendor.
No container bundles NVPTX + AMDGCN + SPIR-V + x86.

80+ papers surveyed — no published system performs cross-vendor MLIR-native runtime dispatch

Our Contribution: `libkdl` — Kernel Dynamic Linker

Six components, ~500 LOC:

1. **Multi-target AOT** — uses existing `gpu-module-to-binary`
2. **Kernel routing table** — maps `(kernel, vendor, arch)` → binary offset
3. **Capability contracts** — each variant declares requirements
4. **Cost-model selection** — roofline-based ranking
5. **Fallback chain** — GPU → GPU → CPU
6. **Device discovery** — dlopen-based, no link-time deps

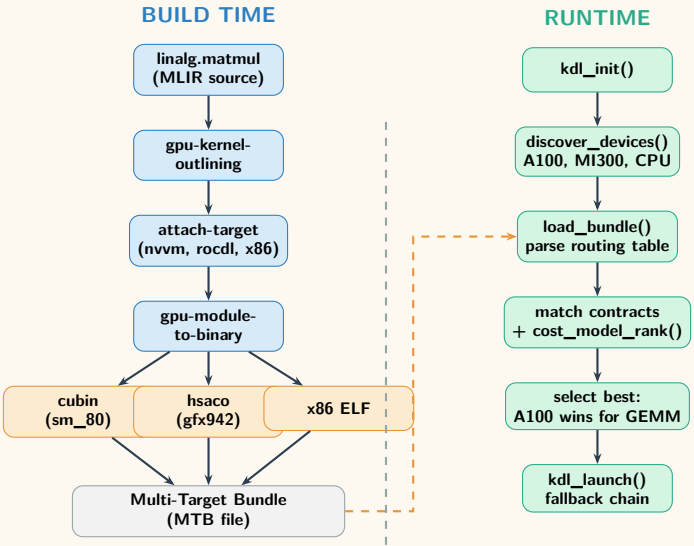
Analogy: `ld.so` for GPU Kernels

- `ld.so` resolves shared libraries at runtime based on system library path
- `libkdl` resolves kernel binaries at runtime based on available GPU hardware

What Makes It Novel

- MLIR-native + runtime
- Cross-vendor + lightweight
- Cost-model-driven
- No programming model req.

Architecture: Build-Time + Runtime Flow



Key API: kdl.h

```
/* Lifecycle */
kdl_status kdl_init(kdl_ctx* out_ctx);
void       kdl_finalize(kdl_ctx ctx);

/* Device Discovery (dlopen-based, no link-time deps) */
int        kdl_get_device_count(kdl_ctx ctx);
kdl_status kdl_get_device_info(kdl_ctx ctx, int idx, kdl_device_info* out);

/* Bundle Loading */
kdl_status kdl_load_bundle(kdl_ctx ctx, const char* path, kdl_bundle_t* out);

/* Kernel Selection (device_index=-1 for auto-select via cost model) */
kdl_status kdl_select_kernel(kdl_ctx ctx, kdl_bundle_t bundle,
                             const char* kernel_name,
                             int device_index, kdl_kernel_t* out);

/* Launch */
kdl_status kdl_launch(kdl_kernel_t kernel,
                      uint32_t grid[3], uint32_t block[3],
                      uint32_t shared_mem, void** args);
kdl_status kdl_sync(kdl_kernel_t kernel);
```

Device discovery via `dlopen("libcuda.so.1")` / `dlopen("libamdhip64.so")` — zero link-time dependencies

Cost Model: Roofline-Based Variant Ranking

Roofline Equation

$$T_{\text{est}}(k, d) = \frac{\text{FLOPS}(k)}{\min\big(\text{PeakFLOPS}(d),\ \text{BW}(d) \cdot I(k)\big)}$$

where $I(k)$ = arithmetic intensity of kernel k

Capability Contract (per variant)

```
gemm_nvptx: requires {
  cuda >= 11.0, sm >= 80,
  shared_mem >= 48KB
}
compute_profile: {
  flops: 2e9,
  bytes_read: 8e6,
  arithmetic_intensity: 250.0
}
```

Selection Flow

1. Check cache (hash lookup), hit = return
2. Find kernel in routing table
3. For each device and variant pair: match contract, estimate cost
4. Pick lowest-cost (device, variant)
5. Load binary via driver API
6. Cache result for future calls

Even a rough roofline model beats random. or priority-only selection. cuBLAS ML recommender: **93%** of optimal.

Results: Dispatch Overhead and Performance

Dispatch Overhead

Path	Latency
Function pointer (ours)	$<$**1 ns**
Full dispatch layer	$<$**10 ns**
CUDA kernel launch	5–20 μs
HIP kernel launch	5–15 μs
Vulkan vkQueueSubmit	10–50 μs
Typical ML kernel	1–10 ms

Dispatch overhead: $<$**0.001%** of ML kernel time

Performance Data Points

Metric	Value
ALPAKA vs native	$>$94%
Kokkos P3 scores	0.75–0.99
SYCL P3 scores	0.46–0.65
SPIR-V vs native	50–80%
AdaptiveCpp JIT	$\sim$15% overhead
Proteus vs AOT	2.8$\times$ gain
Helix mixed GPU	3.3$\times$ throughput

Our dispatch adds $<$**10 ns** on top of **5–20 μs** irreducible launch floor.
1,000–20,000$\times$ smaller.

Integration: torch.compile + ONNX Runtime

torch.compile Backend

```python
@torch._dynamo.register_backend
def kdl_backend(gm, example_inputs):
    mlir = torch_mlir.compile(
        gm, example_inputs,
        output_type="linalg")
    bundle = kdl_compile(mlir)
    ctx = kdl_init()
    bun = kdl_load_bundle(ctx, bundle)

    def dispatch(*args):
        k = kdl_select_kernel(
            ctx, bun, "forward", -1)
        return kdl_launch(k, ...)
    return dispatch
```

ONNX Runtime Execution Provider

```cpp
class KdlExecutionProvider
    : public IExecutionProvider {
  // Claim supported ONNX ops
  GetCapability() -> supported_ops;

  // ONNX -> torch-mlir -> MLIR -> MTB
  Compile() -> multi_target_bundle;

  // Runtime: auto-select best device
  Compute() ->
    kdl_select_kernel() +
    kdl_launch();
};
```

Complementary to IREE

- IREE: full model deployment (100K+ LOC)
- libkdl: add dispatch to *existing* pipelines

Future Work

Near-Term

- **Upstream MLIR integration** — implement custom `OffloadingLLVMTranslationAttrInterface` to replace `#gpu.select_object`
- **Vulkan/SPIR-V backend** — extend `mgpu*` runtime to support `vkQueueSubmit`

Medium-Term

- **Learned cost model** — replace roofline with lightweight ML predictor (like cuBLAS recommender)
- **Shape-dependent multi-versioning** — file-size variants per shape bucket

Long-Term Vision

- **LLVM offload binary format** — propose MTB as extension to existing `0x10FF10AD` format
- **Production torch.compile backend** — full graph-level compilation with fallback
- **Federated kernel registries** — share multi-target bundles across deployments
- **Auto-tuning integration** — combine with Triton/TVM schedule search

Goal: make runtime kernel dispatch as natural as dynamic library linking.

Key References

1. Lattner et al. "MLIR: Scaling Compiler Infrastructure..." CGO 2021
2. IREE Project — Issues #50, #12230, #15334
3. Tillet et al. "Triton..." MAPL 2019
4. Davis et al. "Taking GPU Programming Models to Task..." ICS 2025
5. Pennycook et al. "A Metric for Performance Portability" SC 2016
6. Zenker et al. "Alpaka..." arXiv:1602.08477
7. Georgakoudis et al. "Proteus..." CGO 2025
8. Alpay & Heuveline. "One Pass to Bind Them..." IWOCL 2023
9. Ivanov et al. "Retargeting and Respecializing..." CGO 2024
10. Yang et al. "HetGPU..." arXiv:2506.15993
11. Vasilache et al. "Composable and Modular MLIR Codegen" arXiv:2202.03293
12. Chen et al. "TVM..." OSDI 2018
13. Tristan et al. "Multi-level IR for GPU" arXiv:2507.15277
14. Trott et al. "Kokkos 3" IEEE TPDS 2022
15. Lin et al. "Universal GPU ISA" arXiv:2603.28793
16. "GPU Portability Needs Autotuning" arXiv:2505.03780
17. ONNX Runtime EP Architecture — onnxruntime.ai
18. IRIS — IEEE TPDS 2024
19. Helix (ASPLOS 2025) — Mixed GPU clusters
20. KernelEvolve (Meta) — arXiv:2512.23236

Thank You

Questions?

S Akash

IIT Patna | CERN GSoC | vLLM Contributor

`mlir-hetero-dispatch` / `libkdl`